DA6401 - Introduction to Deep Learning

Assignment-2: Building a Complete Visual Perception Pipeline

General Instructions & Academic Integrity

- This is an individual assignment. Collaborations or discussions with other students are strictly prohibited.
- Tools such as ChatGPT or Claude are permitted **only** as conceptual aids or thought partners. They must not be used to generate the final code submission.
- All submissions will undergo rigorous plagiarism and AI-generated code detection. If plagiarism or unauthorized AI-assisted coding is detected, a grade of **zero** will be assigned.
- Training and test datasets must be strictly isolated. We will verify that the data split was performed properly and randomly.
- Any attempt to artificially inflate accuracy (e.g., data leakage or including test samples in the training set) will result in an immediate grade of **zero** for the entire assignment.
- You must submit a **Public W&B Report**. Ensure the link is accessible to the public during the evaluation phase; **failure to do so will result in a negative marking penalty**.
- **No extensions** will be granted beyond the provided deadline under any circumstances.
- Students are responsible for checking **Moodle** and the official course website for regular updates and clarifications regarding the assignment

Submission Details

- **Release Date:** 21th March 2026, 00:00 AM
- **Submission Deadline:** 12th April 2026, 23:59 PM
- **Late Submission Deadline:** 14th April 2026, 23:59 PM (with penalty)
- **Gradescope:** The formal submission of your code base and W&B report must be completed via **Gradescope**.

Assignment Overview

In this assignment, you will transition to **PyTorch** to build a comprehensive, multi-stage Visual Perception Pipeline. Rather than solving isolated problems, your final deliverable will be a cohesive system capable of detecting, classifying, and segmenting subjects within an image.

- **Permitted Libraries:** `torch`, `numpy`, `matplotlib`, `scikit-learn`, `wandb`, `albumentations`.
- **Project Structure:** Follow the official Assignment-2 GitHub Skeleton `https://github.com/MiRL-IITM/da6401_assignment_2`.

To manage compute resources while achieving this, we will strictly limit the assignment to the following dataset:

Oxford-IIIT Pet Dataset: A rich dataset that provides class labels (breed), bounding boxes (head), and pixel-level masks (trimaps). This will act as the foundation for your perception pipeline. `https://www.robots.ox.ac.uk/~vgg/data/pets/`

1 Implementation & Evaluation Requirements (50 Marks)

1.1 Task 1: VGG11 Classification with Custom Regularization

Implement the VGG11 architecture from scratch using PyTorch and train it for classifing the 37 pet breeds in the dataset. You must adhere to the following specific requirements:

- **VGG11 from Scratch:** Construct the network using standard `torch.nn` modules (e.g., `Conv2d`, `Linear`). The use of pre-built VGG models is strictly prohibited.
- **Batch Normalization:** Integrate `BatchNorm2d` and/or `BatchNorm1d` layers to modernize the architecture.
- **Custom Dropout Layer:** Implement your own Dropout layer by inheriting from `torch.nn.Module`. You are **not** allowed to use `torch.nn.Dropout` or `torch.nn.functional.dropout`.
- **Architectural Reasoning:** You have the freedom to decide the exact placement of your Batch Normalization and Dropout layers. However, you must provide a brief, written theoretical or empirical justification for your design choices.

1.2 Task 2: Encoder-Decoder for Object Localization

Extend your model to perform single-object localization using the bounding box annotations provided in the dataset. Using the VGG11 architecture from Task 1 as your foundational encoder, implement the following:

- **Encoder Adaptation:** Isolate the convolutional backbone from your trained VGG11 model to serve as the feature extractor. You must explicitly state and justify whether you are freezing these pretrained weights or fine-tuning them during this localization task.
- **Regression Decoder:** Design and attach a new regression head to your encoder.
  - **Output:** The head must output exactly four continuous values corresponding to the bounding box coordinates: $[x_{center}, y_{center}, width, height]$.
  - **Scaling:** Ensure your network's output activations are appropriate for your chosen coordinate space.
- **Custom IoU Loss Function:** Implement a custom Intersection over Union (IoU) loss function by inheriting from `torch.nn.Module`. You may not use built-in IoU loss functions from external libraries.

1.3 Task 3: U-Net Style Semantic Segmentation

Extend your architecture to semantic segmentation. You must construct a U-Net inspired network using your VGG11 implementation from Task 1 as the contracting path (encoder). Adhere to the following specifications:

- **Symmetric Decoder:** Implement an expansive path that structurally mirrors your VGG11 encoder to rebuild the spatial resolution of the image.
  - **Learnable Upsampling:** You must use Transposed Convolutions to progressively upsample the spatial dimensions. Standard interpolation algorithms (e.g., bilinear) or unpooling layers are **not** permitted for the primary upsampling steps.
  - **Feature Fusion:** At each stage of the decoder, concatenate the upsampled feature maps with the corresponding, spatially-aligned feature maps from the encoder along the channel dimension before applying the next set of convolutions.
- **Loss Formulation:** Train the network using a suitable objective. You must explicitly state and justify your choice of loss function.

1.4 Task 4: Unified Multi-Task Pipeline

Integrate the components developed in Tasks 1, 2, and 3 into a single, cohesive multi-task learning architecture. Your unified model must adhere to the following specifications:

- **Single Forward Pass:** Implement a unified `forward(self, x)` method that branches from the shared backbone into the three respective task heads. A single forward pass must simultaneously yield:
  1. **Breed Label:** The 37-class classification logits.
  2. **Bounding Box:** The continuous coordinate regression.
  3. **Segmentation Mask:** The dense, pixel-wise spatial map.

Automated Evaluation Pipeline

The submission will be evaluated based on the following weighted criteria. All tasks below are strictly evaluated by an automated grading script:

1. **VGG11 Architecture Verification (5 Marks):** The autograder will instantiate your model and trace a forward pass, strictly checking intermediate feature map dimensions after specific convolutional and pooling blocks to ensure adherence to the standard VGG11 topology.
2. **Custom Dropout Verification (10 Marks):** Your custom dropout `Module` will be isolated and subjected to unit testing. The autograder will pass tensors through the layer at various dropout probabilities (p) to verify the statistical correctness of the binary mask, the application of inverted dropout scaling, and the deterministic behavior when the `self.training` flag is set to `False`.
3. **Custom IoU Loss Verification (5 Marks):** Your custom loss function will be tested against a comprehensive suite of bounding box pairs to verify mathematical accuracy, numerical stability, and gradient viability.
4. **End-to-End Pipeline Evaluation on Private Test Set (30 Marks):** Your unified multi-task pipeline will be executed against a held-out private test set. The network's performance will be quantitatively evaluated using the following metrics:
   - **Classification:** Macro F_1-Score across all 37 pet breed classes.
   - **Detection:** Mean Average Precision (mAP) for the predicted bounding box coordinates.
   - **Segmentation:** Dice Similarity Coefficient (Dice Score) to evaluate pixel-wise mask overlap.

2 Weights & Biases Report (50 Marks)

You must submit a public W&B report link documenting your experiments. Provide robust visualizations, code snippets, and written analyses for the following:

2.1 The Regularization Effect of Dropout (5 Marks)

Train your model with and without Batch Normalization. On a same input, plot the distribution of activations for the 3rd convolutional layer. How did BatchNorm affect the convergence speed and the maximum stable learning rate?

2.2 Internal Dynamics (5 Marks)

Train your model under three conditions: (1) No Dropout, (2) Custom Dropout $p = 0.2$, and (3) Custom Dropout $p = 0.5$. Overlay the Training vs. Validation Loss curves for all three runs in W&B. Explain how your custom dropout implementation successfully alters the generalization gap.

2.3 Transfer Learning Showdown (10 Marks)

Evaluate the impact of different transfer learning strategies by comparing their performance on the semantic segmentation pipeline. You must run, track, and log experiments using Weights & Biases (W&B) for the following three distinct approaches:

- **Strict Feature Extractor:** Freeze the entire VGG11 pre-trained backbone. Only train the newly initialized symmetric decoder and task heads.
- **Partial Fine-Tuning:** Freeze the early convolutional blocks (which capture generic, low-level features like edges and gradients), but unfreeze the last one or two convolutional blocks alongside your decoder.
- **Full Fine-Tuning:** Unfreeze the entire network and update all weights end-to-end.

In your report, provide a comprehensive discussion addressing the following points:

- **Empirical Comparison:** Include and compare your W&B validation curves (Dice Score / Accuracy and Loss) for all three strategies. Discuss the observed differences in convergence speed, training stability, computational time per epoch, and final validation performance. Which strategy ultimately performed best?
- **Theoretical Justification:** Explain *why* your empirical results occurred. Theoretically, why does unfreezing the later convolutional blocks (or the entire model) perform better compared to using the network as a strictly frozen feature extractor?

2.4 Inside the Black Box: Feature Maps (5 Marks)

Pass a single image of a dog through your trained classification model from Task 1. Extract and visualize the feature maps from the *first* convolutional layer and the *last* convolutional layer before the pooling layer. What differences do you observe in the visual patterns? How does the network transition from learning localized edges to high-level semantic shapes (like snouts or ears)?

2.5 Object Detection: Confidence & IoU (5 Marks)

Log a table in W&B containing at least 10 test images from the Pet dataset with bounding box predictions overlaid. Color code the boxes: Green for Ground Truth, Red for Predictions. Display the Confidence Score and the calculated Intersection over Union (IoU) for each predicted box in the table. Identify a failure case (an image with high confidence but low IoU, or a completely missed object). What aspects of the image (occlusion, scale, complex background) confused the model?

2.6 Segmentation Evaluation: Dice vs. Pixel Accuracy (5 Marks)

For your U-Net style model, log 5 sample images showing: (1) Original Image, (2) Ground Truth Trimap, (3) Predicted Trimap Mask. Track both **Pixel Accuracy** and **Dice Score** during validation. You will likely observe that Pixel Accuracy appears artificially high compared to the Dice Score, especially in early epochs. Explain this phenomenon mathematically based on the distribution of pixels (foreground vs. background) in the dataset. Why is the Dice Coefficient a superior metric for highly imbalanced segmentation tasks?

2.7 The Final Pipeline Showcase (5 Marks)

In your W&B report, upload the final output images from your pipeline running on 3 completely novel images of pets downloaded from the internet (not from the dataset). Briefly evaluate how well your pipeline generalized to these "in-the-wild" images. Did the bounding box accurately crop the subject for the classifier? Did the U-Net struggle with non-standard lighting or backgrounds?

2.8 Meta-Analysis and Reflection (10 Marks)

Conclude your report with a comprehensive meta-analysis of your unified multi-task pipeline. You must maintain a Weights & Biases (W&B) project logging all experimental phases.

- **Comprehensive Metric Plots:** Generate and embed clear plots comparing **Training vs. Validation (or Testing)** performance across your training history. You must include overlaid plots for *all* primary metrics and losses for *each* task.
- **Retrospective Architectural Reflection:** Write a technical summary evaluating how your earlier isolated design decisions impacted the final unified pipeline.
    1. **Architectural Reasoning (Revisiting Task 1):** How did the specific placement and configuration of your Custom Dropout and Batch Normalization layers impact the multi-task network?
    2. **Encoder Adaptation (Revisiting Task 2):** Reflect on your strategy for the VGG11 backbone (frozen vs. fine-tuned). Did your shared backbone suffer from task interference?
    3. **Loss Formulation (Revisiting Tasks 3):** Evaluate the effectiveness of your chosen segmentation loss.